

SLAM : Projet Agora Sprint 5, demande 1**Création d'une base de données,
création et gestion d'une entité
avec le composant Doctrine****Auteur :** Yannick Narbey**Pré-requis :**

- Symfony : contrôleurs, routes, templates twig
- Agora Sprint 4 : Créer une application et un contrôleur avec Symfony

T A B L E D E S M A T I E R E S

1.	Contexte et objectifs	2
2.	Mise en place des prérequis techniques	3
2.1	Installation des modules Symfony complémentaires	3
2.2	Configuration et création de la nouvelle base de données	4
3.	Création de l'entité et de la table Genre	6
3.1	Point sur l'architecture du projet	6
3.2	Création de l'entité genre	6
3.3	Création de la table genre	7
4.	Opération de création, modification et suppression avec doctrine	10
4.1	Créer un Genre	10
4.2	Lire un genre	11
4.3	Modifier un genre	16
4.4	Supprimer un genre	17
5.	Conclusion	19

1. Contexte et objectifs

L'association Agora souhaite, dans le sprint suivant, pouvoir donner des **droits d'accès différents aux utilisateurs** en fonction de leur rôle dans l'association et cela au travers d'une **interface pour gérer les comptes d'accès**.

L'administration des données de l'application doit donc évoluer à terme pour s'appuyer sur les mécanismes de sécurité natifs de Symfony.

Pour cela, il devient nécessaire de reconstruire l'ensemble des formulaires à l'aide de l'ORM Doctrine qui seront pris en charge nativement par le composant de sécurité de Symfony.



Symfony (rappel)

Symfony est un ensemble de composants PHP, un **framework d'application Web**, une philosophie et une communauté - tous travaillant ensemble en harmonie. (site officiel)

Symfony est un framework basé sur le modèle MVC qui permet de réaliser des sites complexes rapidement, mais de façon structurée et avec un code clair et maintenable.

SensioLabs est le créateur du framework PHP Symfony. Le nom SensioLabs vient du mot espagnol "sencillo" qui signifie "simple". Fabien Potencier est le fondateur et chef de projet Symfony.



<https://symfony.com/>



Doctrine

Symfony est livré par défaut avec l'**ORM** Doctrine. Un ORM (pour **Object-Relation Mapper**, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des **correspondances** entre les **schémas de la base** de données et les **classes** du programme applicatif.



<https://www.doctrine-project.org>



Entité

Les **données** de la base de données sont des **objets**. Un objet dont l'enregistrement est confié à l'ORM se nomme une **entité** (entity en anglais). On dit également **persister une entité**, plutôt qu'enregistrer une entité.



Attributs PHP et Doctrine

Doctrine utilise les attributs PHP 8 afin de définir la table correspondant à une entité, définir un identifiant (id) en auto-incrément, nommer les colonnes, etc. Ces informations se nomment les metadata de l'entité.

Ajouter les metadata à un objet, s'appelle **mapper l'objet**, c'est-à-dire faire le lien entre l'objet et la représentation physique qu'utilise Doctrine (une table de la base de données relationnelle).

Le développement de l'application se fera dans un cadre de travail agile **SCRUM**.

2. Mise en place des prérequis techniques

2.1 Installation des modules Symfony complémentaires

Les composants dont dépend l'application sont répertoriés dans le fichier **composer.json** placé à la racine de l'application. Pour les besoins du sprint 5, il faut ajouter le composant doctrine.

Saisir la commande suivante pour installer l'ORM doctrine qui va gérer la BDD MySQL :

```
C:\wamp64\www\mielapp>composer require symfony/orm-pack
```

```
C:\Windows\System32\cmd.e x + v
C:\wamp64\www\agoram50>composer require symfony/orm-pack
./composer.json has been updated
Running composer update symfony/orm-pack
Loading composer repositories with package information
Restricting packages listed in "symfony/symfony" to "6.0.*"
Updating dependencies
Lock file operations: 17 installs, 0 updates, 0 removals
- Locking doctrine/cache (2.2.0)
- Locking doctrine/collections (2.4.0)
- Locking doctrine/common (3.5.0)
- Locking doctrine/dbal (3.10.3)
- Locking doctrine/doctrine-bundle (2.13.0)
- Locking doctrine/doctrine-migrations-bundle (3.5.0)
- Locking doctrine/event-manager (2.0.1)
- Locking doctrine/instantiator (2.0.0)
- Locking doctrine/migrations (3.5.5)
- Locking doctrine/orm (2.20.7)
- Locking doctrine/persistence (3.4.3)
- Locking doctrine/sql-formatter (1.5.3)
- Locking friendsofphp/proxy-manager-lts (v1.0.19)
- Locking laminas/laminas-code (4.16.0)
- Locking symfony/doctrine-bridge (v6.0.19)
- Locking symfony/orm-pack (v2.4.1)
- Locking symfony/polyfill-php84 (v1.33.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 17 installs, 0 updates, 0 removals
- Downloading doctrine/collections (2.4.0)
- Downloading doctrine/persistence (3.4.3)
- Downloading friendsofphp/proxy-manager-lts (v1.0.19)
- Downloading doctrine/sql-formatter (1.5.3)
- Downloading doctrine/orm (2.20.7)
- Downloading doctrine/doctrine-migrations-bundle (3.5.0)
- Installing doctrine/cache (2.2.0): Extracting archive
- Installing symfony/polyfill-php84 (v1.33.0): Extracting archive
- Installing doctrine/collections (2.4.0): Extracting archive
- Installing doctrine/event-manager (2.0.1): Extracting archive
```

```
Class PDOJeux located in ./src/Controller/modele/class.PDOJeux.inc.php does not comply with psr-4 autoloading standard (rule: App\ => ./src). Skipping.
56 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Run composer recipes at any time to see the status of your Symfony recipes.

Executing script cache:clear [OK]
Executing script assets:install public [OK]

Found 7 security vulnerability advisories affecting 5 packages.
Run "composer audit" for a full list of advisories.

What's next?

Some files have been created and/or updated to configure your new packages.
Please review, edit and commit them: these files are yours.

doctrine/doctrine-bundle instructions:

* Modify your DATABASE_URL config in .env
* Configure the driver (postgresql) and
  server_version (16) in config/packages/doctrine.yaml

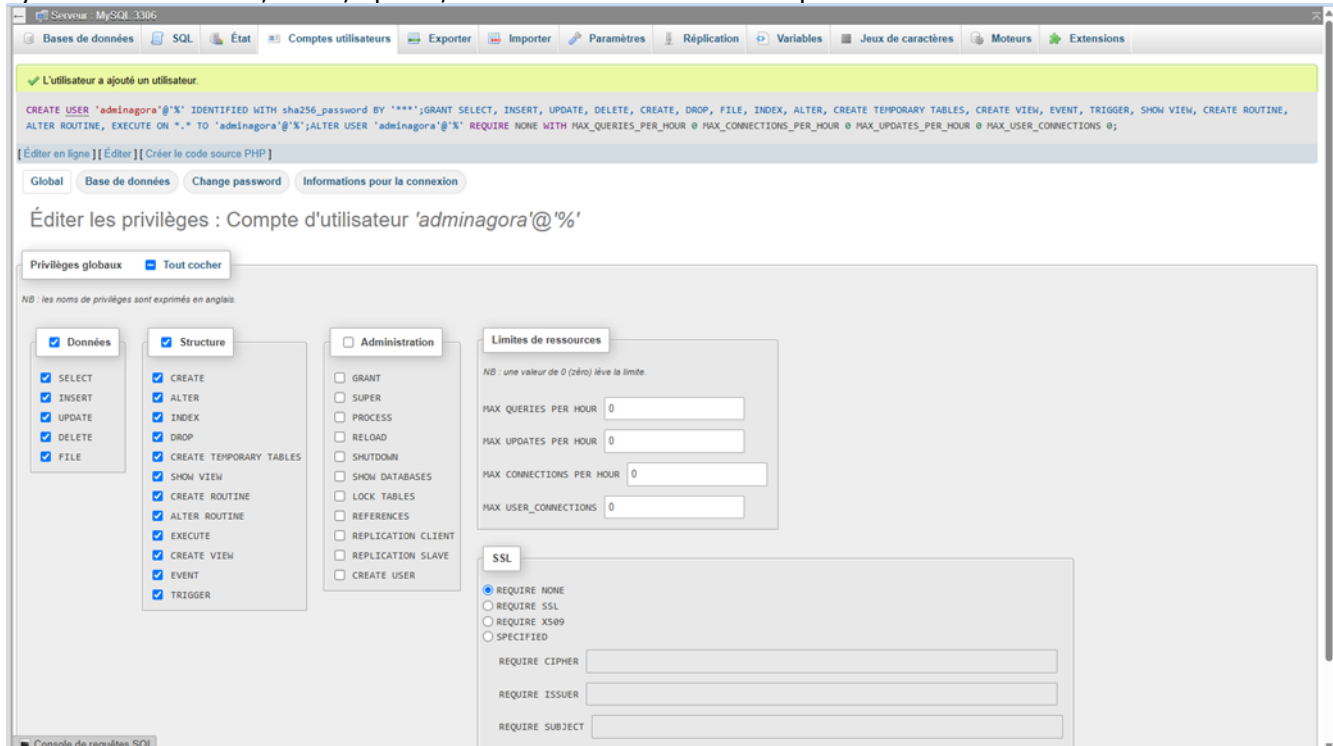
Found 7 security vulnerability advisories affecting 5 packages.
Run "composer audit" for a full list of advisories.
```

Si vous n'utilisez pas docker, répondre « n » à la question
Do you want to include Docker configuration from recipes?

2.2 Configuration et création de la nouvelle base de données

Création du compte dédié à l'application

Créer un compte adminagora avec un mdp présentant un niveau de complexité suffisant (par exemple Agora1234 !) ayant les droits select, insert, update, delete sur les données ainsi que les droits « structure ».



Les informations pour la connexion à la base de données sont stockées dans une variable d'environnement nommé : **DATABASE_URL**.

Personnaliser cette variable dans le fichier **.env** à la racine du répertoire du projet:

Fichier agora52/.env

```

. . .

###> doctrine/doctrine-bundle ###
# Format described at https://www.doctrine-project.org/projects/doctrine-dbal/en/latest/reference/configuration.html#connecting-using-a-url
# IMPORTANT: You MUST configure your server version, either here or in config/packages/doctrine.yaml
#
# DATABASE_URL="sqlite:///kernel.project_dir%/var/data.db"
#
DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=8.0.32&charset=utf8mb4"
# DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=10.11.2-MariaDB&charset=utf8mb4"
DATABASE_URL=mysql://adminagora:Agora1234!@127.0.0.1:3306/agoraorm?serverVersion=9.0.0
#DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=15&charset=utf8"
###< doctrine/doctrine-bundle ###
. . .

```

Adminagora:Agora12345! sont les credential du compte avec privilege ayant le droit de créer la base de données de l'application

127.0.0.1:3306 est l'IP et le port d'administration du serveur MySQL que l'ORM doctrine va utiliser pour créer la base de données.

agoraorm est le nom de la base de données qui va être générée avec doctrine.

Renseigner :

- l'identifiant:mot_de_passe de la base de données,
- la version de MySql installée sur le poste.



ATTENTION, dans le DSN, nom d'utilisateur et mot de passe, évitez les caractères considérés comme spéciaux dans un URL (comme +, @, \$, #, /, :, *, !) (voir [RFC 3986](https://symfony.com/doc/current/doctrine.html#installing-doctrine) pour une liste complète) sinon il convient de les encoder (<https://symfony.com/doc/current/doctrine.html#installing-doctrine>)

Doctrine peut créer la base de données, à partir de ces paramètres de configuration, avec la commande :

```
C:\wamp64\www\agora5> php bin/console doctrine:database:create
```

```
Created database `agoraorm` for connection named default
```

```
C:\wamp64\www\agora5>
```

Remarque :

La commande **php bin/console list doctrine** affiche la liste des commandes doctrine.

3. Création de l'entité et de la table Genre

3.1 Point sur l'architecture du projet

A ce point du projet, la nouvelle base de données agoraorm est vide, l'ensemble des données est dans la base de données agora gérée par les méthodes créées lors du sprint précédent.

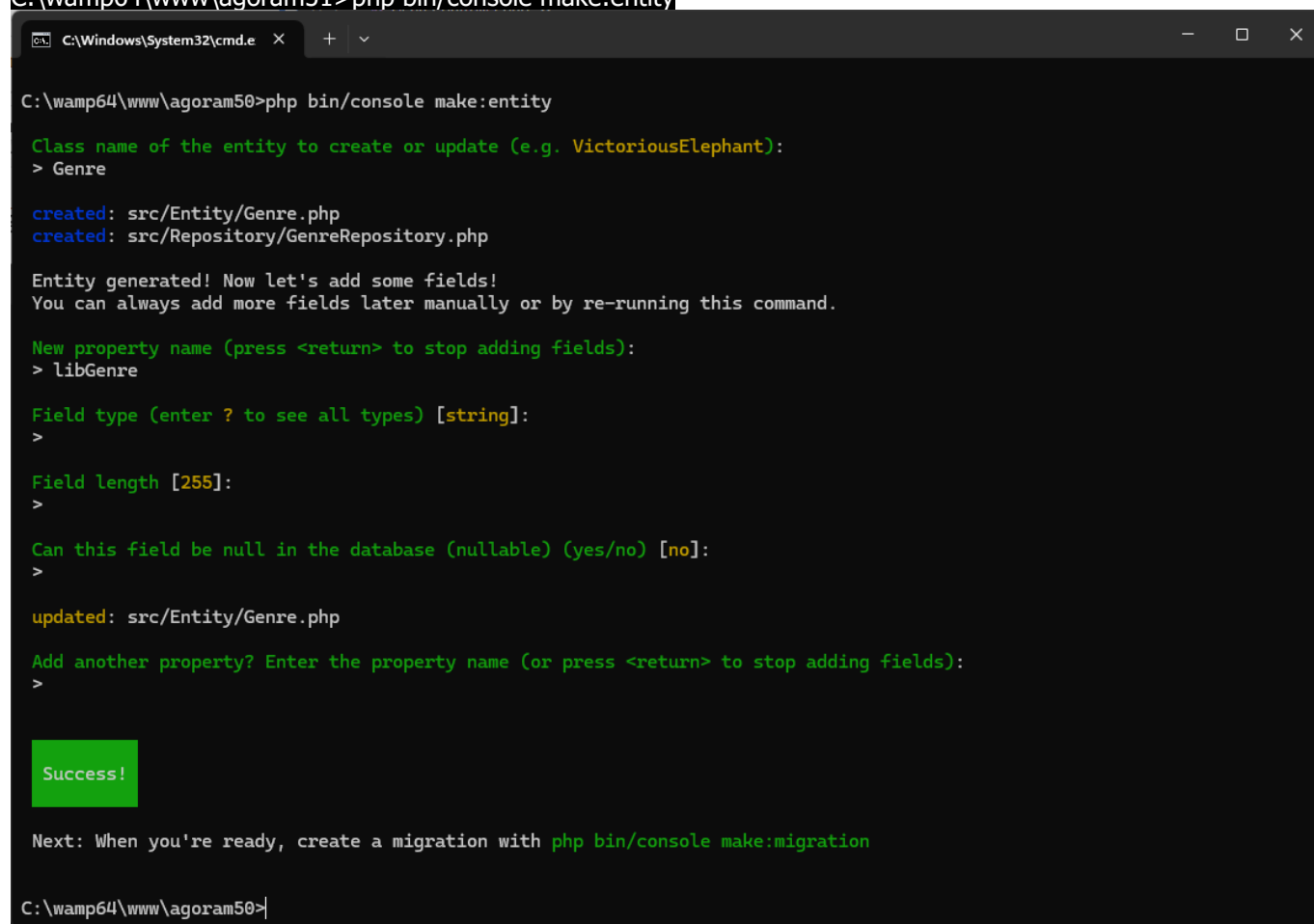
L'objectif va donc être de reconstruire les formulaires, en tirant partie des fonctionnalités de l'ORM. Dans un premier temps, nous allons créer les contrôleurs en mesure de gérer les genres :

- Créer les genres,
- Afficher les genres,
- Modifier les genres,
- Supprimer les genres.

3.2 Création de l'entité genre

Arrêter le serveur Symfony (Ctrl+C), puis lancer la commande permettant de créer l'entité Genre

C:\wamp64\www\agoram51>php bin/console make:entity



```
C:\Windows\System32\cmd.exe
C:\wamp64\www\agoram50>php bin/console make:entity

Class name of the entity to create or update (e.g. VictoriousElephant):
> Genre

created: src/Entity/Genre.php
created: src/Repository/GenreRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.

New property name (press <return> to stop adding fields):
> libGenre

Field type (enter ? to see all types) [string]:
>

Field length [255]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Genre.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

Success!

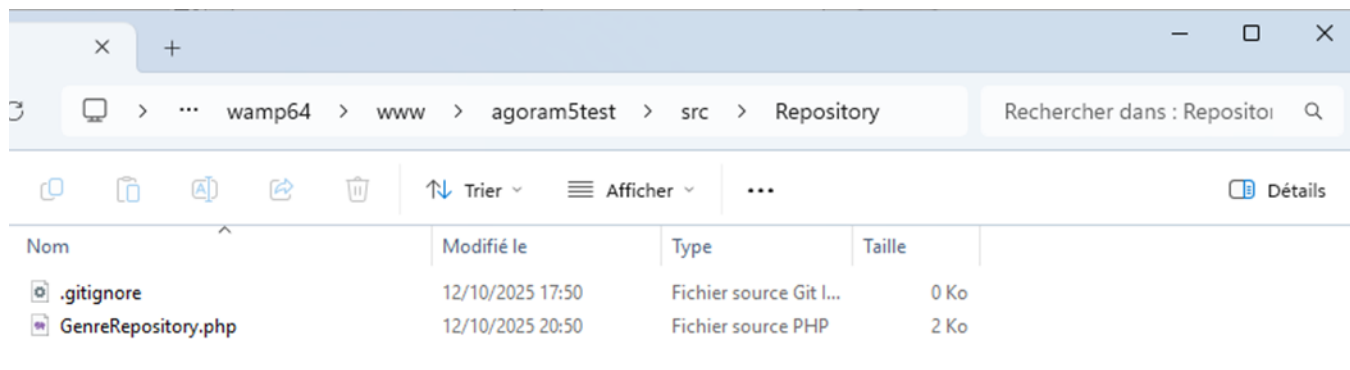
Next: When you're ready, create a migration with php bin/console make:migration

C:\wamp64\www\agoram50>
```

Doctrine générera automatiquement l'id auto-incrémental de l'entité Genre.

Symfony va générer le fichier suivant :

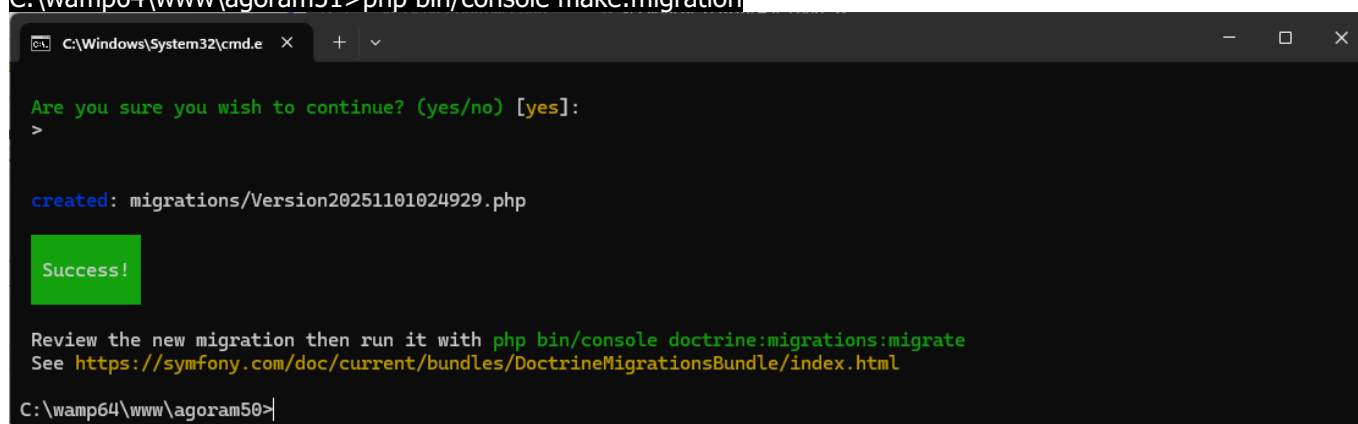
- src/Repository/GenreRepository.php



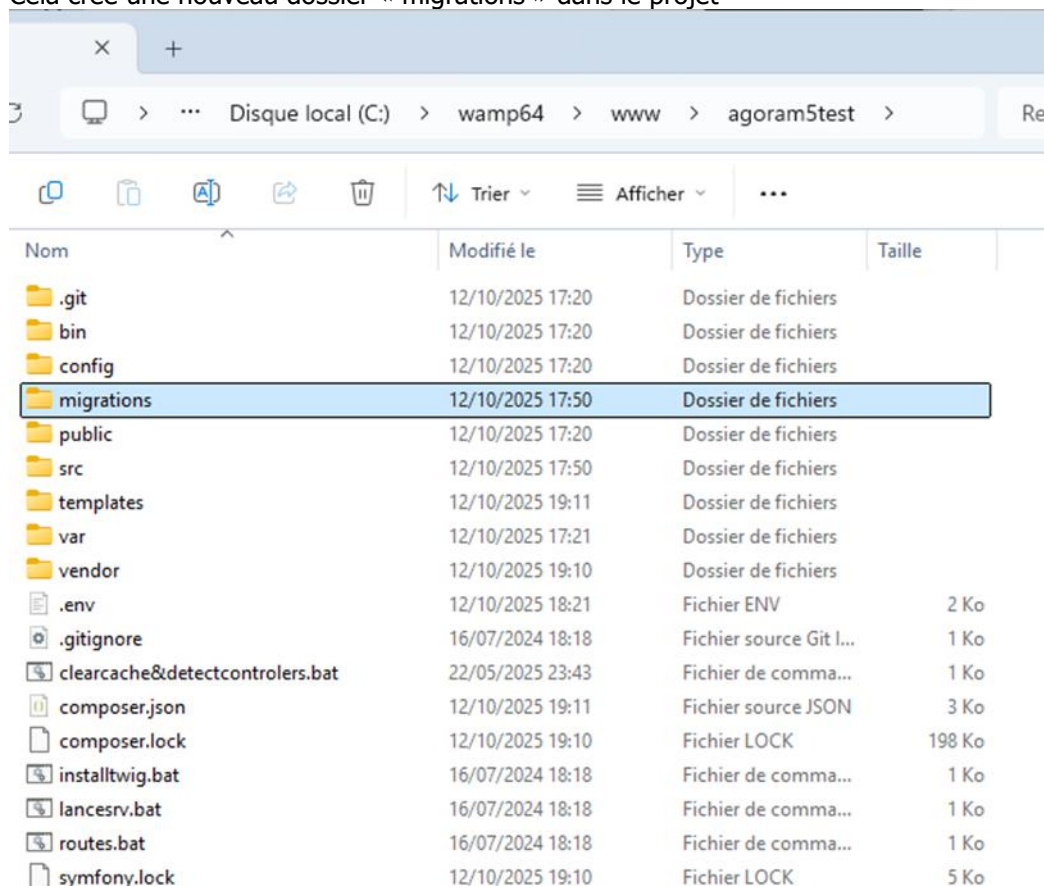
3.3 Création de la table genre

Lancer la migration permettant de créer la table correspondant à cette nouvelle entité :

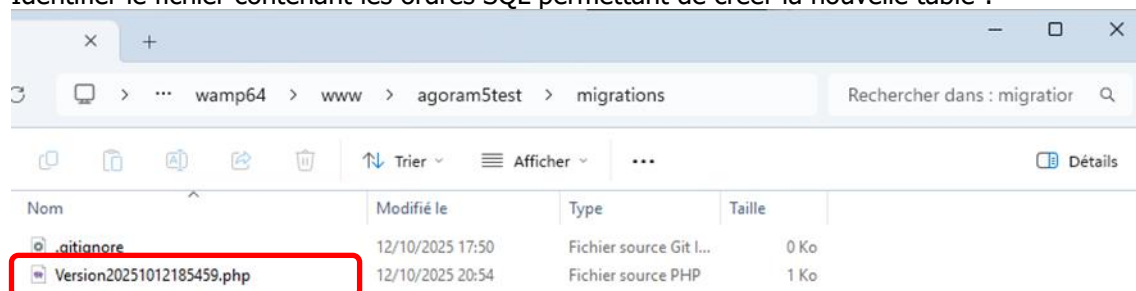
C:\wamp64\www\agoram51>php bin/console make:migration



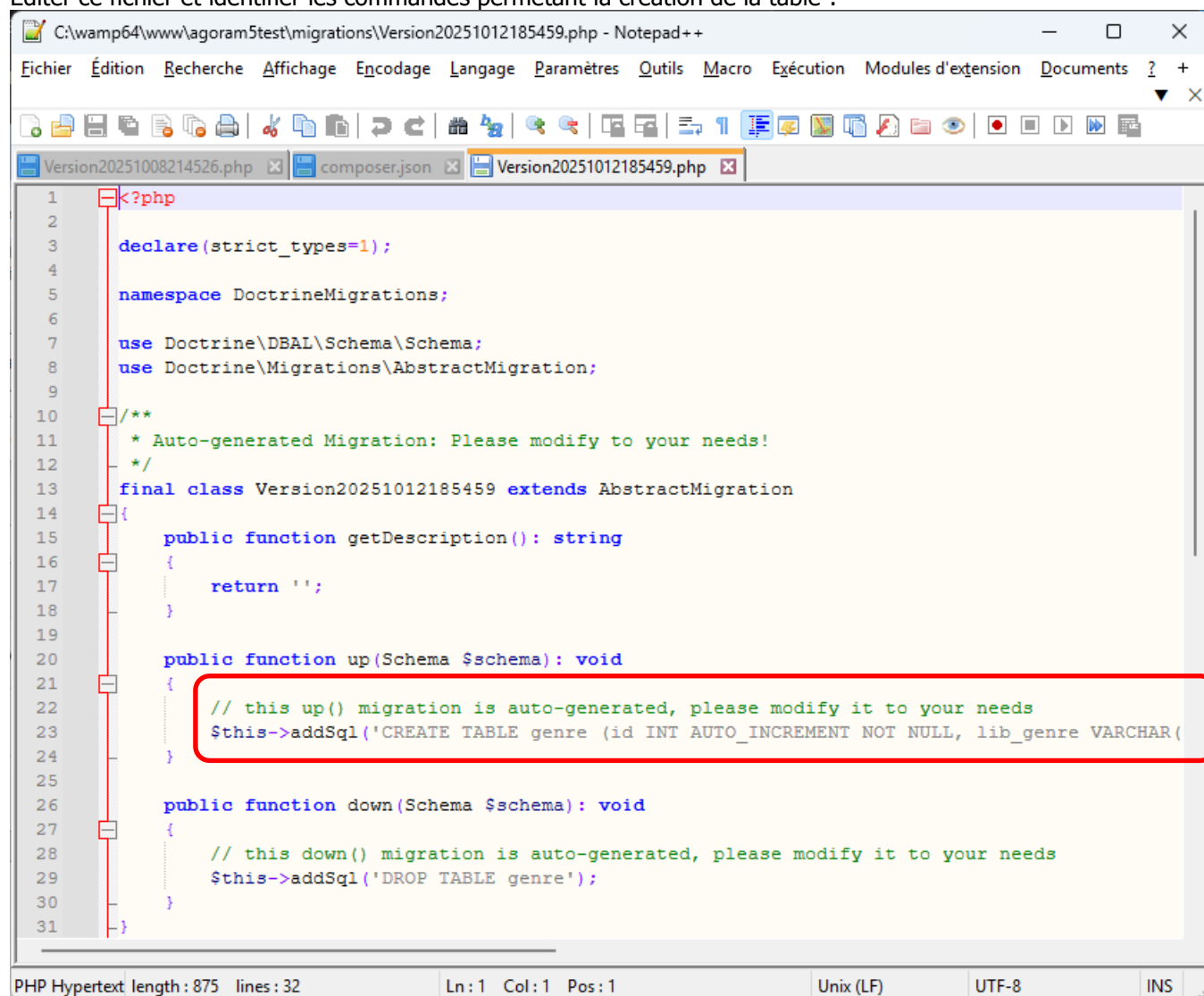
Cela crée un nouveau dossier « migrations » dans le projet



Identifier le fichier contenant les ordres SQL permettant de créer la nouvelle table :

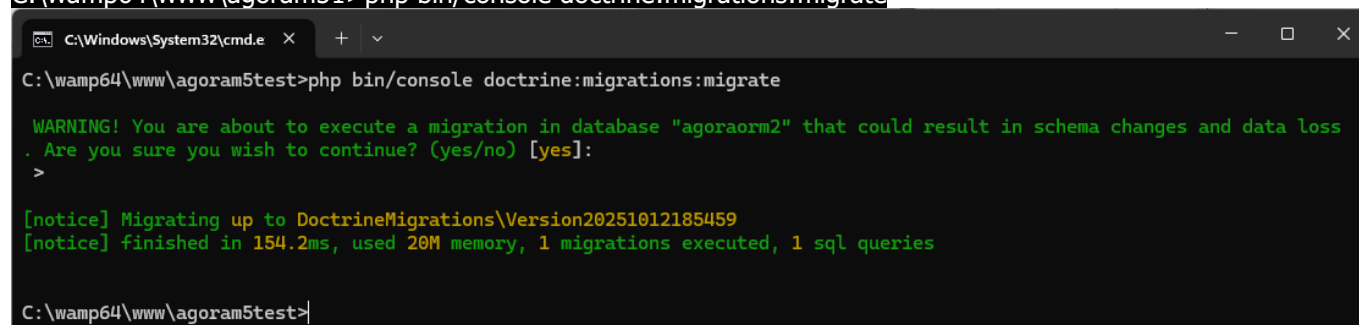


Editer ce fichier et identifier les commandes permettant la creation de la table :



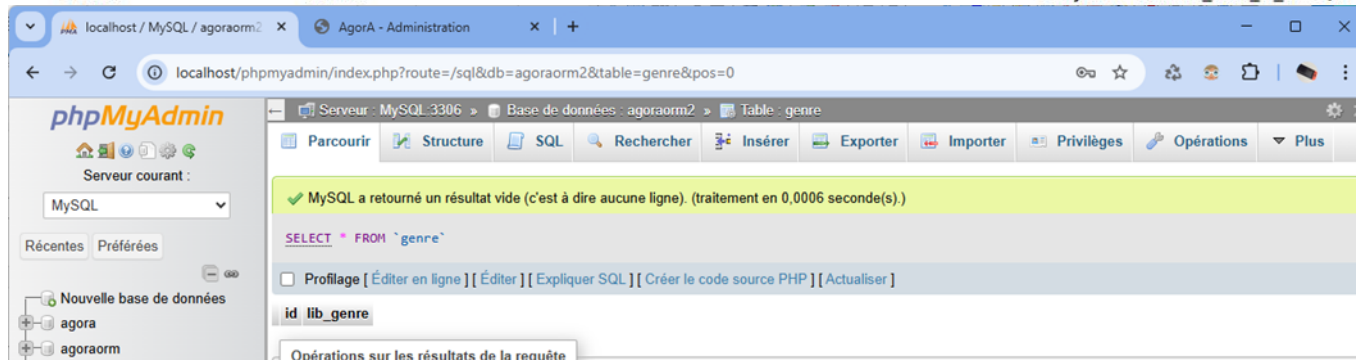
Une fois le fichier vérifié, lancer la commande suivante pour appliquer la migration :

C:\wamp64\www\agoram51>php bin/console doctrine:migrations:migrate



Vérifier la création, dans PHPMyAdmin, de la table genre correspondant à l'entité :

Table	Action	Lignes	Type	Interclassement	Taille
☐ doctrine_migration_versions	★ Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb3_unicode_ci	16,0
☐ genre	★ Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_unicode_ci	16,0
2 tables	Somme	1	MyISAM	utf8mb4_0900_ai_ci	32,0



localhost / MySQL / agoraorm2 x AgorA - Administration x | +

localhost/phpmyadmin/index.php?route=/sql&db=agoraorm2&table=genre&pos=0

phpMyAdmin

Serveur courant : MySQL

Récentes Préférences

Nouvelle base de données

agora

agoraorm

Table : genre

Parcourir Structure SQL Rechercher Insérer Exporter Importer Privileges Opérations Plus

MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0006 seconde(s).)

SELECT * FROM `genre`

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

id lib_genre

Opérations sur les résultats de la requête

4. Opération de création, modification et suppression avec doctrine

4.1 Créer un Genre

1. Démarrez le serveur Symfony dans un nouveau terminal :

```
symfony server:start
```

ou

```
symfony serve --no-tls
```

2. Modifions le code pour créer un nouveau genre et l'enregistrer dans la base de données. On crée l'objet Genre et ensuite on persiste l'objet. Lire attentivement les commentaires.

Fichier agora51/src/Controller/GenreController.php

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Entity\Genre;
use Doctrine\ORM\EntityManagerInterface;

class GenreController extends AbstractController
{
    #[Route('/genre', name: 'app_genre')]
    public function index(): Response
    {
        return $this->render('genre/index.html.twig', [
            'controller_name' => 'GenreController', 'menuActif' => 'Tournois',
        ]);
    }

    #[Route('/genre/creer', name: 'app_genre_creer')]
    public function creerGenre(EntityManagerInterface $entityManager): Response
    {
        // : Response          type de retour de la méthode creerGenre
        // pour récupérer le EntityManager (manager d'entités, d'objets)
        // ajout de l'argument à la méthode comme ici creerGenre(EntityManagerInterface $entityManager)
        // ou on peut récupérer le EntityManager comme dans la méthode suivante

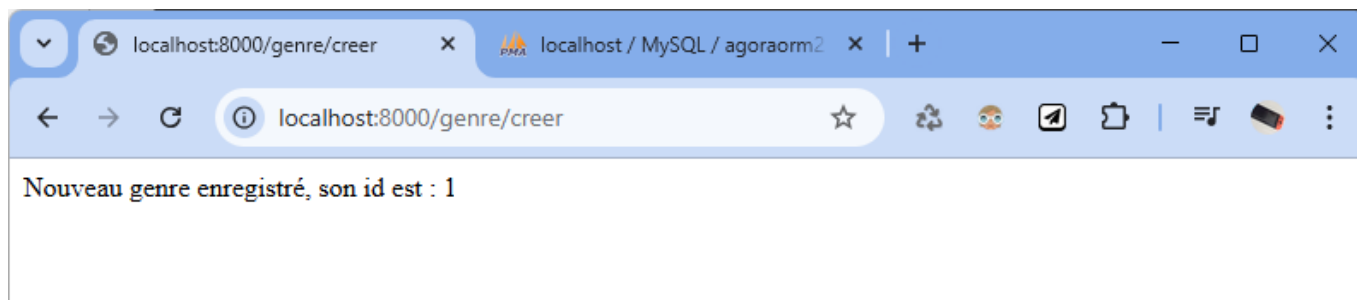
        // créer l'objet
        $genre = new Genre();
        $genre->setLibGenre('Aventure');

        // dire à Doctrine que l'objet sera (éventuellement) persisté
        $entityManager->persist($genre);

        // exécuter les requêtes (indiquées avec persist) ici il s'agit de l'ordre INSERT qui sera exécuté
        $entityManager->flush();

        return new Response("Nouveau genre enregistré, son id est : " . $genre->getId());
    }
}
```

3. Vérifiez que le serveur MySQL tourne et testez avec `http://localhost:8000/genre/creer`



4. Vérifiez le contenu de la base avec la commande suivante. Sous Powershell, remplacez les " par des ' :

```
C:\wamp64\www\agora51>php bin/console doctrine:query:sql "SELECT * FROM tournoi"
```

```
-----  
id  lib_genre  
-----
```

```
1  Aventure  
-----
```

4.2 Lire un genre

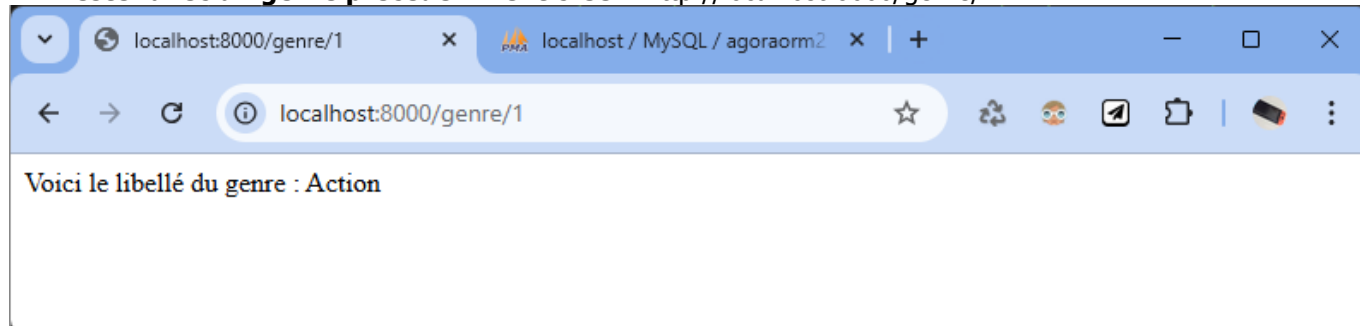
1. Ajoutez l'import de ManagerRegistry et la méthode lire() dans le contrôleur pour récupérer un tournoi à partir de son id

Fichier agora51/src/Controller/GenreController.php

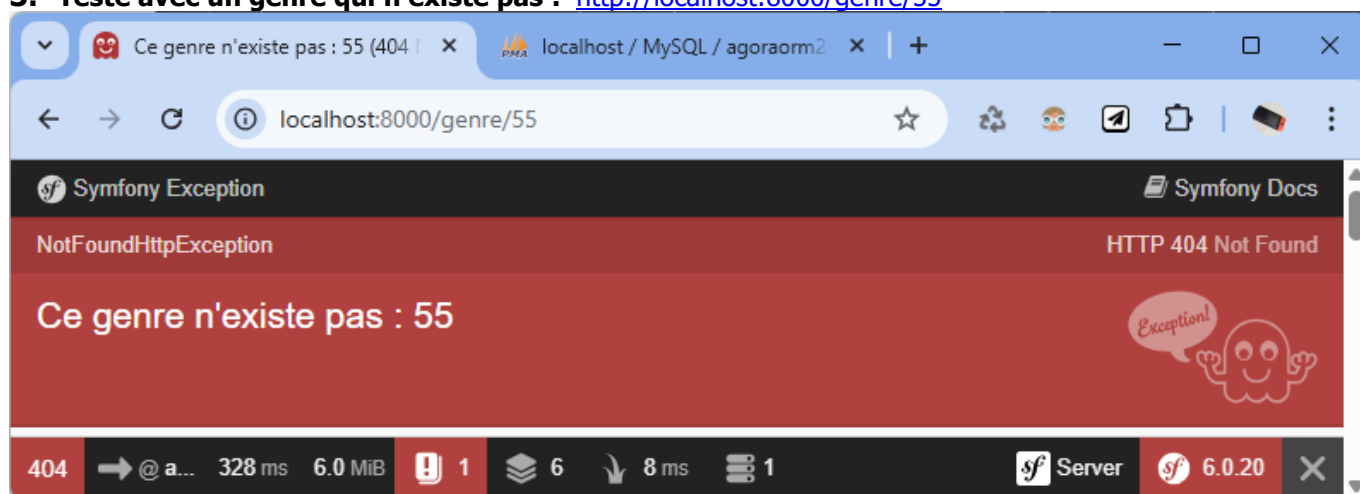
```
use Doctrine\Persistence\ManagerRegistry;  
  
#[Route('/genre/{id}', name: 'app_genre_lire')]  
public function lire($id, ManagerRegistry $doctrine)  
{  
    // ces 2 exemples retournent le entity manager par défaut  
    // ici nous n'utilisons qu'une base de données donc le entity manager par défaut suffit  
    $entityManager = $doctrine->getManager();  
    // $entityManager = $doctrine->getManager('default');  
  
    // {id} dans la route permet de récupérer $id en argument de la méthode  
    // on utilise le Repository de la classe Genre  
    // il s'agit d'une classe qui est utilisée pour les recherches d'entités (et donc de données dans la base)  
    // la classe GenreRepository a été créée en même temps que l'entité par le make  
  
    $genre = $entityManager  
        ->getRepository(Genre::class)  
        ->find($id);  
  
    if (!$genre) {  
        throw $this->createNotFoundException(  
            'Ce genre n\'existe pas : ' . $id  
        );  
    }  
  
    return new Response("Voici le libellé du genre : " . $genre->getLibGenre());  
}
```

```
// on peut bien sûr également rendre un template
}
```

2. Tester avec un genre précédemment créé : <http://localhost:8000/genre/1>



3. Teste avec un genre qui n'existe pas : <http://localhost:8000/genre/55>



Le message d'erreur ci-dessous apparaît naturellement quand l'ID mentionné dans l'URL ne correspond à aucun genre, puis que la commande ci-dessous est exécutée quand \$genre n'existe pas :

```
if (!$genre) {
    throw $this->createNotFoundException(
        'Ce genre n\'existe pas : ' . $id
    );
}
```

4. Noter que la classe Genre référence son repository :

Fichier agora51/src/Entity/Genre.php

```
<?php

namespace App\Entity;

use App\Repository\GenreRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: GenreRepository::class)]
class Genre
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
```

```
#[ORM\Column]
private ?int $id = null;
```

L'annotation #[ORM\Entity(repositoryClass: GenreRepository::class)] indique à **Doctrine** :

"Pour toutes les opérations de persistance ou de recherche concernant cette entité Genre, utilise la classe GenreRepository." qui est **responsable des requêtes** et de la logique d'accès aux données pour l'entité Genre.

Un **repository** est une **abstraction de la base de données**. Il sert à :

- Trouver des entités (find, findAll, findBy, etc.)
- Écrire des requêtes personnalisées (avec le QueryBuilder ou DQL)
- Encapsuler la logique d'accès aux données pour éviter de la disperser dans le code

5. Autres méthodes disponibles en standard grâce au repository :

Fichier agora51/src/Controller/GenreController.php
Ce sont des exemples, ne les copiez pas dans le code

```
$repository = $entityManager->getRepository(Genre::class);

// cherche un genre à partir de sa clé primaire (en général "id")
$genre = $repository->find($id);

// cherche un genre à partir de son libellé
$genre = $repository->findOneBy(['libGenre' => 'simulation de vol']);
// cherche un genre à partir de son id et de son libellé
$genre = $repository->findOneBy([
    'ID' => 1,
    'libGenre' => 'aventure'
]);

// cherche les genres correspondant au libellé, triés par libellé
$genre = $repository->findBy(
    ['libGenre' => 'aventure'],
    ['libGenre' => 'ASC']
);

// cherche tous les genres
$genre = $repository->findAll();
```

6. Observer le contenu du repository.

Bien que seule la méthode __construct() ne soit rédigée dans cette classe, des méthodes disponibles additionnelles sont indiquées dans les **annotations**, car elle sont héritées de ServiceEntityRepository :

- find()
- findOneBy()
- findAll()
- findBy ()

Fichier agora51/src/Repository/GenreRepository.php

```
<?php

namespace App\Repository;

use App\Entity\Genre;
```

```
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepository;
use Doctrine\Persistence\ManagerRegistry;

/**
 * @extends ServiceEntityRepository<Genre>
 * @method Genre|null find($id, $lockMode = null, $lockVersion = null)
 * @method Genre|null findOneBy(array $criteria, array $orderBy = null)
 * @method Genre[]    findAll()
 * @method Genre[]    findBy(array $criteria, array $orderBy = null, $limit = null, $offset = null)
 */

class GenreRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Genre::class);
    }

    /**
     * @return Genre[] Returns an array of Tournoi objects
     */
    public function findByExampleField($value): array
    {
        return $this->createQueryBuilder('g')
            ->andWhere('g.exampleField = :val')
            ->setParameter('val', $value)
            ->orderBy('g.id', 'ASC')
            ->setMaxResults(10)
            ->getQuery()
            ->getResult()
            ;
    }

    public function findOneBySomeField($value): ?Genre
    {
        return $this->createQueryBuilder('g')
            ->andWhere('g.exampleField = :val')
            ->setParameter('val', $value)
            ->getQuery()
            ->getOneOrNullResult()
            ;
    }
}
```

Nous verrons par la suite comment ajouter des méthodes personnalisées dans le repository pour les besoins plus spécifiques.

7. Autre façon de lire le genre

Dans le contrôleur ci-dessous, les informations récupérées sur le genre passé en paramètre, le sont au moyen du getter `getLibGenre()`

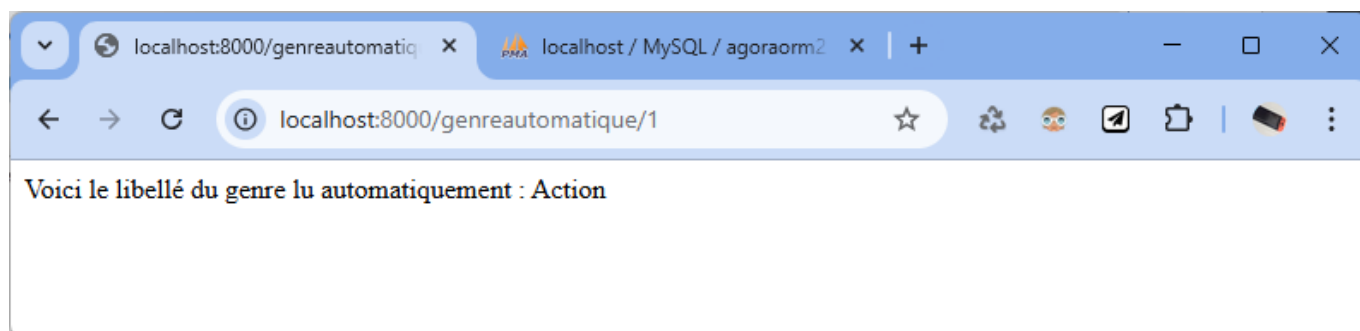
Fichier `agora51/src/Controller/GenreController.php`

...

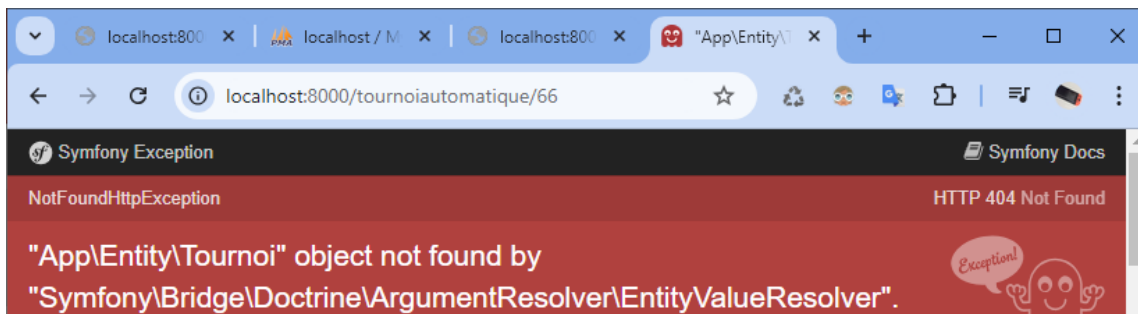
```
#[Route('/genreautomatique/{id}', name: 'app_genreautomatique_lire')]
public function lireautomatique(Genre $genre)
{
    // grâce au Symfony\Bridge\Doctrine\ArgumentResolver\EntityValueResolver
    // il suffit de donner le genre en argument
    // la requête de recherche sera automatique
    // et une page 404 sera générée si le genre n'existe pas

    return new Response('Voici le libellé du genre lu automatiquement : '
        . $genre->getLibGenre()) ;
    // on peut bien sûr également rendre un template
}
```

8. Tester avec genre précédemment créé : <http://localhost:8000/genreautomatique/1>



9. Tester avec un genre inexistant : <http://localhost:8000/genreautomatique/66>



"Symfony\Bridge\Doctrine\ArgumentResolver\EntityValueResolver"

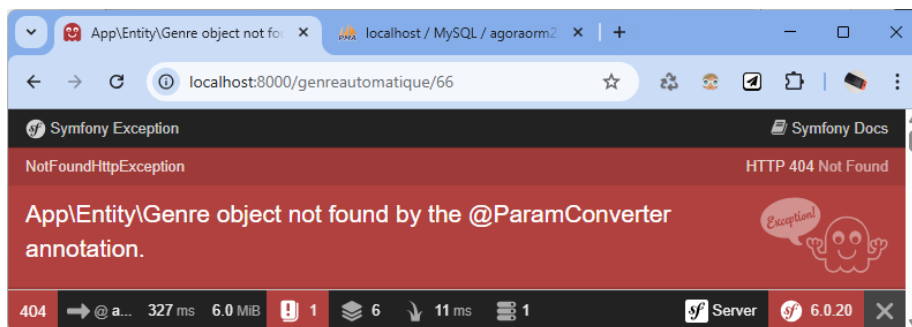
Les bridges permettent de faire des ponts avec des projets open-source comme Doctrine ou Twig.

À chaque fois que vous utilisez sur Twig la méthode path pour générer le chemin d'une route, ce n'est pas twig/twig qui fournit cette méthode mais symfony/twig-bridge. Sur Symfony 6.2, il existe des Bridge pour Doctrine, Monolog, PHPUnit, ProxyManager et Twig.

Le **EntityValueResolver** de Doctrine Bridge remplace le ParamConverter du SensioFrameworkExtraBundle qui a été abandonné.

Plus d'informations ici : https://symfony.com/doc/current/controller/value_resolver.html

ou



Symfony essaye de convertir le paramètre {id} de la route en objet genre. Puisque dans ce cas, il ne trouve pas de genre 66, il obtient null et comme le contrôleur attend un objet Genre et pas null, ParamConverter déclenche une exception automatique «App\entity\Genre object not found by the @Paramconvert annotation ».

4.3 Modifier un genre

1. Ajoutez une méthode pour modifier un genre

Fichier agora51/src/Controller/GenreController.php

```
#[Route('/genre/modifier/{id}', name: 'app_genre_modifier')]
public function modifier($id, EntityManagerInterface $entityManager)
{
    // 1 recherche du genre
    $genre = $entityManager->getRepository(Genre::class)->find($id);

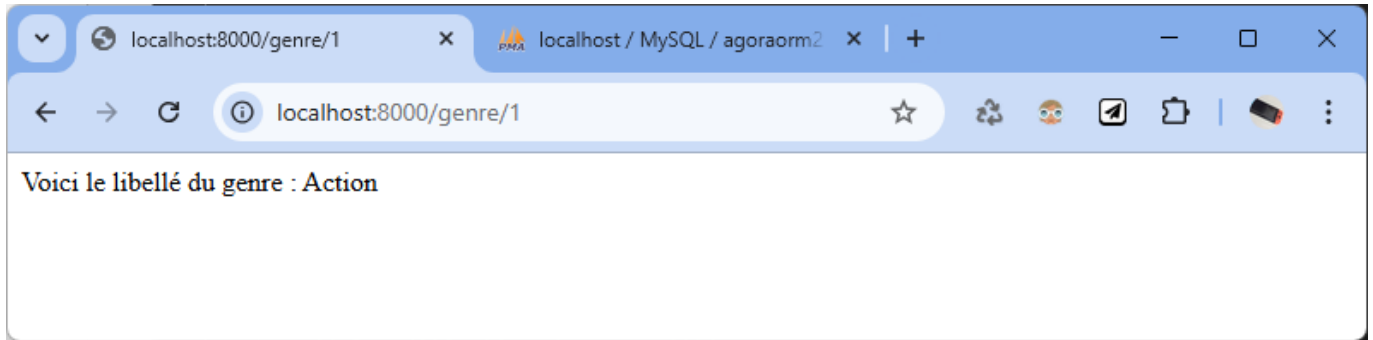
    // en cas de genre inexistant, affichage page 404
    if (!$genre) {
        throw $this->createNotFoundException(
            'Aucun genre avec l\'id ' . $id
        );
    }

    // 2 modification des propriétés
    $genre->setLibGenre('Action');
    // 3 exécution de l'update
    $entityManager->flush();

    // redirection vers l'affichage du genre
    return $this->redirectToRoute('app_genre_lire', [
        'id' => $genre->getId()
    ]);
}
```

Noter que le contrôleur opère la modification au moyen de l'appel d'un setter setLibGenre() suivi de flush() qui exécute la mise à jour.

2. Tester : <http://localhost:8000/genre/modifier/1>



3. Vérifiez dans la base :

```
C:\wamp64\www\agora51>php bin/console doctrine:query:sql "SELECT * FROM genre"
-----
id  lib_genre
-----
1   Action
-----
```

4.4 Supprimer un genre

1. Ajoutez une méthode pour supprimer un genre

Fichier agora51/src/Controller/GenreController.php

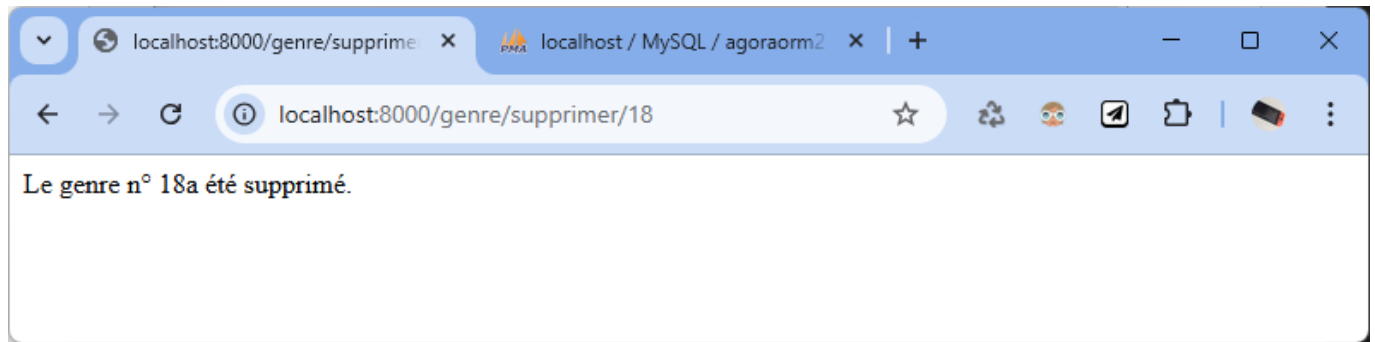
```
#[Route('/genre/supprimer/{id}', name: 'app_genre_supprimer')]
public function supprimer($id, EntityManagerInterface $entityManager)
{
    // 1 recherche du genre
    $genre = $entityManager->getRepository(Genre::class)->find($id);

    // en cas de genre inexistant, affichage page 404
    if (!$genre) {
        throw $this->createNotFoundException(
            'Aucun genre avec l\'id ' . $id
        );
    }

    // 2 suppression du genre
    $entityManager->remove(($genre));
    // 3 exécution du delete
    $entityManager->flush();

    // affichage réponse
    return new Response('Le genre n° ' . $id . ' a été supprimé.');
```

2. Tester la suppression du genre précédemment créé : <http://localhost:8000/genre/supprimer/1>



3. Vérifier le succès de la suppression en base de données :

```
C:\wamp64\www\agora51>php bin/console doctrine:query:sql "SELECT * FROM genre"
-----
id  lib_genre
-----
-----
```

5. Conclusion

Voilà un premier aperçu de l'utilisation de l'ORM Doctrine.

Vous savez désormais comment :

- Créer une base de données
- créer une entité,
- migrer cette entité en base de données
- puis créer les contrôleurs en mesure de gérer cette entité.

Nous verrons à la prochaine étape comment automatiser l'ensemble de ces opérations.